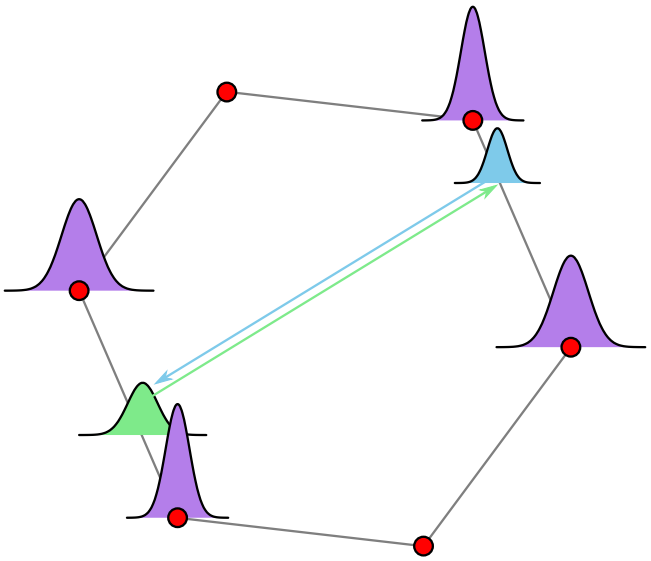


Final Report

The Art of Scientific Computing: Project Title: A selection of granary queueing models

Author:
William Batchelor

Subject Handbook code:
COMP-90072



Faculty of Science
The University of Melbourne

Original Version: 1st March 2018

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

Chapter 1

Introduction

Granaries are storage structures for grain, used to protect harvested grain from poor weather and pests. To prevent spoilage grain needs to be kept at a low moisture content; this state may initially be achieved by harvesting during dry conditions, or by drying harvested grain. Drying sometimes happens at the granary. It is typical for a granary to service all of the farms in an area, often operating either as a collective or under commercial agreements **Hassanizadeh**.

Grain must be harvested at the correct time of year, and the need for dry conditions leads to busy peak periods for granaries corresponding with appropriate weather. Deliveries cluster, as farmers restart harvesting operations after inclement weather or race to harvest as much as possible before rain.

These high demand periods can lead to significant queues, incurring costs of time and money as trucks sit idle and farms run out of buffer storage. Understanding the likely demands placed on a granary is therefore important to decisions regarding its design and operation. Planners may wish to invest in equipment to reduce unloading time or add additional service bays; managers may wish to alter policy regarding opening hours or the order in which trucks are served. Mathematical models are a useful way of investigating these demands and evaluating different investments and policies citeberruto 2000, Kryukova, berruto.

In this investigation we implement three simulation models of a granary, and investigate where the models agree and disagree. We compare to closed form results for queueing systems, and consider where the models likely fall short of reality.

Chapter 2

Background

This section derived from Sztrik 2016.

Before considering the specifics of granaries, it is worth establishing some basic queueing theory.

In the typical terminology, a customer is an entity subject to a queue, and a server is an entity that provides some resource to customers. In the usual case a queue is FIFO - First In First Out, a system familiar to anyone who has visited a grocery store. It is worth being aware of an alternative structure, the priority queue, where customers have a priority value, with servers always serving the next highest priority customer.

Queueing systems are classified with Kendall's notation, a shorthand to quickly communicate the behaviour of the system. A classification is shown as A/S/c, where A denotes the probability distribution driving customer arrival, S denotes the distribution driving service time, and c denotes the number of servers in the system.

We will consider some examples to get an idea of the scheme.

A D/D/1 system has deterministic arrivals, deterministic service time, and a single server. In other words, customers always show up the same amount of time after the previous customer, and they are always served in a fixed amount of time. This system is of little interest, if the server processes customers faster than they arrive there is never a queue, and if it is slower the queue grows without bound.

An M/D/1 system has Poisson arrivals, deterministic service times, and a single server. M stands for memoryless; Poisson arrivals means the time between arrivals is an exponential random variable.

An M/M/1 system possesses Poisson arrivals, exponentially distributed service times, and a single server.

An M/M/c system possesses Poisson arrivals, exponentially distributed service times, and c servers.

The nice thing about the above systems is they possess simple service functions and memoryless (Poisson) arrival functions, permitting simple closed form solutions to the behaviour of the queue in the long term.

A G/D/1 queue on the other hand, with a general (arbitrary) distribution for arrivals, does not necessarily have a closed form solution, meaning analysis must rely on approximation or simulation. For completeness it is worth noting that M/G/1 queues do possess closed form results; memoryless arrival is the important condition for closed form solutions to exist.

M/D/1 queue behaviour can be predicted in the long run analytically using the Pollaczek-Khinchine formula.

$$W_q = \frac{\rho D}{2(1-\rho)}, \qquad L_q = \lambda W_q = \frac{\rho^2}{2(1-\rho)}, \tag{2.1}$$

Where:

λ is the mean arrival rate;

D is the service time;

ρ is the utilisation, the fraction of the time the server will be busy

W_q is the average wait time for a customer;

L_q is the time-averaged queue length;

P_0 is the probability that an arriving customer finds the server idle and so does not wait. Because Poisson arrivals see time averages, this equals the long-run fraction of time the server is idle.

We can go further than just average behaviour. For the M/D/1 queue, the full distribution of the waiting time W is also known in closed form, a result due to Erlang **Franx**:

$$P(W \le t) = (1-\rho) \sum_{k=0}^{\lfloor t/D \rfloor} \frac{[\lambda(kD-t)]^k}{k!} e^{-\lambda(kD-t)}, \qquad t \ge 0, \tag{2.2}$$

giving the probability that a customer waits no longer than t before being served, with $W_q = \mathbb{E}[W]$.

Lastly worth mentioning Little's law as another classic result that we'll lean on for verification. Essentially a conservation law, it demands that the long-run average number of customers in a system equals the rate at which customers arrive multiplied by the average time each one spends there:

$$L = \lambda W, \tag{2.3}$$

where L is the time-averaged number of customers in the system and W is the average time a customer spends in it. This law makes no assumptions about distributions, the number of servers, or the type of queueing. This makes it a good check in situations where we lack nice closed form results.

Chapter 3

Models

Before discussing the three specific models, we will touch on their commonalities, and the behaviour we want to capture. All models feature a single FIFO queue at the granary determining service order. Further, the granary possesses a single server with deterministic service time.

Shared between all three models in one weather and moisture system. This system generates synthetic hourly rainfall data, and then uses that to predict crop moisture minute by minute, which the individual models may then consult to determine their behaviour. Synthetic generation is achieved by defining the expected length of wet and dry periods and toggling between them based on the draw of parameterised geometric distributions, then each hour of wet periods drawing an exponentially distributed value for rainfall. This is translated to moisture by defining maximum and minimum moistures, and stepping up and down linearly towards them based on time of day and rainfall intensity.

3.1 Open model

The first model at its simplest it is an M/D/1 queueing system, with Poisson arrivals.

Interarrival time is driven by a parameter λ, trucks arrive, queue, unload at the granary, before disappearing into the aether.

We can run the simulation in this simple form to confirm against closed form results, but in practice we want to incorporate opening hours and weather effects.

To that end, we track the moisture level in the grain throughout the season. Any trucks arriving while moisture is above some threshold value are rejected with linearly increasing probability. At a certain point, all arrivals are rejected. This has the effect of increasing the interarrival time as conditions become unfavourable to the harvest. Similarly, if the granary is closed for the night, all arrivals are rejected.

3.2 Closed model with abstract farm

The second model increases fidelity by closing the loop and no longer consigning trucks to the void. Instead, we introduce a new entity with its own queue, the farm. Grain accumulates at the farm as it is harvested, trucks queue there to pick up grain, travel to the granary to unload it, before returning to the farm to queue again.

There is a fixed fleet of trucks available, each truck possesses its own travel time between the granary and the farm. Travel times are drawn uniformly from a circle, accounting for the fact that area of farmland available increases as the square of the distance from the granary. Each trip noise is applied to the travel time reflecting variance in conditions on the road.

Again we must deal with weather and opening hours.

Weather enters this model through the rate at which grain is harvested. Similarly to the open model, below a moisture threshold harvest proceeds at full speed, above it the rate decreases linearly until it eventually reaches zero.

Opening hours enter this model in the step where trucks attempt to depart the farm and head to the granary. The decision was made that farms would operate continuously except as prevented by moisture levels, whereas the granary keeps to its opening hours, except for serving trucks in its queue at closing time, which we allow as overtime. To that end, trucks expected to arrive after closing time are instead scheduled to arrive the next morning, with some travel noise. This matches morning clustering often seen in reality.

We also wish to promote clustering after a stoppage due to rainfall. To that end, when such an event occurs, trucks are prohibited from leaving the farm until a threshold of stored grain is reached. The intention here is to reflect that this farm is intended as an abstraction of a number of farms. If all of the farms possess between them only enough grain to fill a single truck, no truck will be filled. Assuming they harvest at a similar rate,

all of them would be expected to release trucks at similar times; thus, the threshold. Away from stoppages it's assumed that if the abstract farm has grain to dispatch then the hypothetical farm any given truck is serving would have grain to dispatch after its round trip. In the case of trucks with short round trips that may not be a good assumption.

Another flaw of this model is the bottleneck around truck dispatch. If the time it takes to fill an individual truck is set to a plausible level, then with a single server the rate at which grain can be dispatched is limited compared to a model with multiple farms, each with their own server. A better model might feature multiple servers drawing from the farms queue, but this was not implemented.

3.3 Closed model with farms

Our third model partakes of most of the characteristics of the second model. The crucial difference is this time we consider multiple farms with their own fleet of trucks and their own queue; the other differences all follow from this.

Travel time to and from each farm is drawn uniformly from a circle, each truck in a farms fleet possesses the same travel time. Noise is still applied independently to each trip.

Weather is treated the same, except each farm of course has its own grain. Each farm experiences the same weather. We no longer need to enforce a threshold of grain before releasing trucks after a stoppage, each farm will naturally wait until it has sufficient grain to fill a truck before dispatching one, producing the desired clustering.

Chapter 4

Implementation

This section will convey the implementation as it stands, alongside some details of the journey to get there.

The initial intention was to use SimPy, a Python library for discrete event simulation. This progressed so far as building a prototype for the first model using the library. To determine whether the library would work a test run of 100 long seasons was run, and found to take around 15 seconds, which was slower than desired given the desired parameter sweep. Profiling with cProfile revealed a substantial amount of the runtime was spent on initialising the simulation, I decided to handroll my own discrete event simulator.

Initially I built a class holding state information and defined functions that acted upon a state object, but as the project grew and I wanted to model several situations this grew unwieldy. I decided to go for an object-oriented approach, defining a parent class holding fundamental data and methods, and having child classes with their own data and methods to implement specific models.

The core of the simulation is a priority queue of events, stored as a binary heap managed by Python's heapq, where priority is decided by scheduled time, with ties broken by an insertion counter. The simulator draws the next element from the event queue, updates the time, and calls whatever method was passed with the event.

Once the first model was implemented with the handrolled system a similar test ran in 1 second, a significant improvement.

The classes and their behaviour are as follows

- Weather: Stores weather parameters. Generates hourly rainfall for the season, then converts to grain moisture at each minute.

- Truck: Stores fields describing an individual truck.

- BasicState: Stores the event priority queue, the granary queue, and methods to run the simulation loop and for trucks arriving at, being served by, and departing the granary. Also reports on visits to and state of the granary, for later analysis.

- OpenState: Child of BasicState. Contains fields and methods necessary to implement Model 1. Has fields and behaviour allowing weather and open hours to be ignored for verification.

- Farm: Contains a fleet of trucks, a farm queue, and fields and methods required to manage that queue.

- ClosedState: Child of BasicState. Contains a list of farms, and the fields and methods required to implement Models 2 and 3. Model 2 is Model 3 with a single farm and different parameters reflecting the abstraction of all farms. Notably the restart threshold is much higher for Model 2, and its truck fleet has a spread of travel times. Possesses fields and behaviour allowing weather and open hours to be ignored for verification.

Seeding for randomisation follows recommended NumPy practices, a root SeedSequence spawns one child per replication, and each replication creates its own generator with default_rng. Weather recieves seperate seeding to the other classes. Weather is generated outside of the simulation loop and passed in, so different parameters can be compared against the same weather.

Experiments are parallelised with joblib. This is straightforward as replications are entirely independent, so the problem is embarrassingly parallel. As replications are seeded by index rather than worker, results are identical regardless of the number of workers or job order.

Chapter 5

Validation

- Weather graph TODO - Include image

First we consider our weather generator. Across 20 harvest long iterations we seem to be averaging around 1mm of rain in a day, give or take fifty percent, spread relatively evenly but lumpily over the duration. By physical intuition this seems about right. Given we aren't modeling a particular system and the relationship with moisture is itself by fiat, this seems good enough for our purposes.

		Mean wait (min)			Prob. no wait		
λ	ρ	Sim.	SEM	Theory	Sim.	SEM	Theory
0.08	0.3	0.85	0.01	0.86	0.703	0.003	0.700
0.13	0.5	1.97	0.02	2.00	0.502	0.003	0.500
0.18	0.7	4.52	0.06	4.67	0.303	0.002	0.300
0.23	0.9	16.81	0.56	18.00	0.104	0.002	0.100

Table 5.1: Verification of Model 1 against M/D/1 closed-form results, with weather and opening hours disabled.

Here we compare the behaviour of Model 1 against closed form expectations, finding strong agreement. Service time $D = 4$ minutes, so $\rho = 4\lambda$. SEM is the standard error of the mean across replications, note that simulated values are typically within one or two SEM of theory, with disagreement growing somewhat at higher ρ values.

TODO - Include graphic

Here we plot simulated wait time distribution against Erlang's closed form result, and observe strong agreement across the whole range.

It appears as though the simulation and at least the open model is behaving appropriately.

For Models 2 and 3 there aren't closed form results to compare against. We can use Little's law to check for bookkeeping errors, and examine queue length over time against its expected behaviour. This information can be found in the next chapter.

Chapter 6

Results and discussion

	Trucks/day		Mean wait		95th pct. wait	
λ	Mean	SEM	Mean	SEM	Mean	SEM
0.05	23.90	0.25	0.49	0.01	3.39	0.03
0.1	47.94	0.49	1.27	0.01	5.95	0.04
0.15	72.03	0.76	2.88	0.02	10.88	0.10
0.2	95.83	0.94	7.00	0.07	23.22	0.35

Table 6.1: Model 1 results. Arrival rate λ in trucks per minute; waits in minutes.

		Trucks/day		Mean wait		95th pct. wait	
Trucks	Harvest rate	Mean	SEM	Mean	SEM	Mean	SEM
10	0.5	24.92	0.17	6.30	0.15	25.95	0.25
10	1	49.73	0.34	4.11	0.15	22.28	0.23
10	2	79.11	2.57	3.44	0.11	18.38	0.26
20	0.5	24.88	0.17	17.30	0.14	59.04	0.14
20	1	49.86	0.34	17.39	0.39	60.16	0.15
20	2	99.26	0.69	12.89	0.53	52.17	0.29
40	0.5	24.89	0.17	17.61	0.12	63.39	0.15
40	1	49.81	0.34	33.43	0.20	110.78	0.34
40	2	99.32	0.69	54.97	0.61	129.93	0.24

Table 6.2: Model 2 results. Waits in minutes.

			Trucks/day		Mean wait		95th pct. wait	
Farms	Trucks/farm	Harvest rate	Mean	SEM	Mean	SEM	Mean	SEM
5	1	0.05	12.40	0.08	2.36	0.08	9.97	0.24
5	1	0.1	24.77	0.17	1.41	0.06	8.27	0.22
5	4	0.05	12.40	0.08	6.80	0.07	26.08	0.28
5	4	0.1	24.86	0.17	13.95	0.09	46.14	0.23
20	1	0.05	49.58	0.34	20.30	0.33	60.28	0.16
20	1	0.1	98.66	0.66	14.03	0.58	52.52	0.26
20	4	0.05	49.58	0.34	40.31	0.28	127.10	0.17
20	4	0.1	99.26	0.67	85.91	0.35	211.02	0.49

Table 6.3: Model 3 results. Waits in minutes.

Tables 6.1-3 show the results of a sweep over model parameters, over three weather systems with eight replications each. This lets us compare mean and tail wait times for the different models at comparable

throughputs. Using this we can choose parameter settings for the next experiment, where we observe queue length over time for each model with comparable settings.

Worth noting here that for Model 3 the number of farms multiplies the harvest rate to get overall harvest rate. Comparison with model 2 is appropriate when the product of those numbers matches Model 2's harvest rate. Note that when this occurs, throughput matches.

It's interesting to note that for Model 3 rows with 5 farms, wait time is lower than comparable entries for Model 2, whereas when we jump to 20 farms wait time is consistently higher than comparable entries for Model 2. I don't have a good model of why this is a case, but my intuition is that at a certain point more farms leads to more severe clustering and thus higher wait times. However, this doesn't explain the decrease in wait time when increasing the number of farms from 1 farm in Model 2 to 5 farms in Model 3.

- Comparison results TODO - Include graph Looking at Figure 6.TODO we see a few things.

Across all models we see periodic periods without queueing. This is good, it means the granary is shutting down for the night as intended.

Model 1 sees queues spread throughout the active period, without any real trend towards clustering. This makes sense for a memoryless model.

Model 2 sees sharp spikes when the granary opens, with some smaller spikes afterwards, and then little queueing throughout the day. Matches our desire for clustering, but reflects a weakness of the model. The throughput bottleneck on the single farm is the same as on the granary, and so I suspect after the initial spike the granary releases trucks at a certain rate, and those trucks return a rate the granary can handle, thus there's little queueing.

Model 3 sees sharp initial spikes followed by second still significant spikes later in the day. This time with multiple farms the return rate isn't bottlenecked, and so the spike can stick around.

Chapter 7

Conclusion

TODO

Bibliography

Sztrik, János (2016). “Basic Queueing Theory”. en. In.